Programming for Finance

SQL
Filtering Data & Combining Tables

Department of Economics

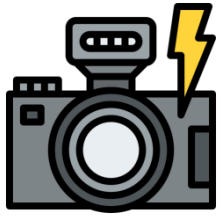
Ivan Bondarev

MGIMO UNIVERSITY

Email

- i.bondarev@inno.mgimo.ru

Signs



– Please take pictures of the information. Medium priority.

Source: https://www.flaticon.com/ru/free–icon/camera–flash_3460307

– Please write down the information. High priority.

Source: https://openmoji.org/library/emoji–270D/

TOPICS

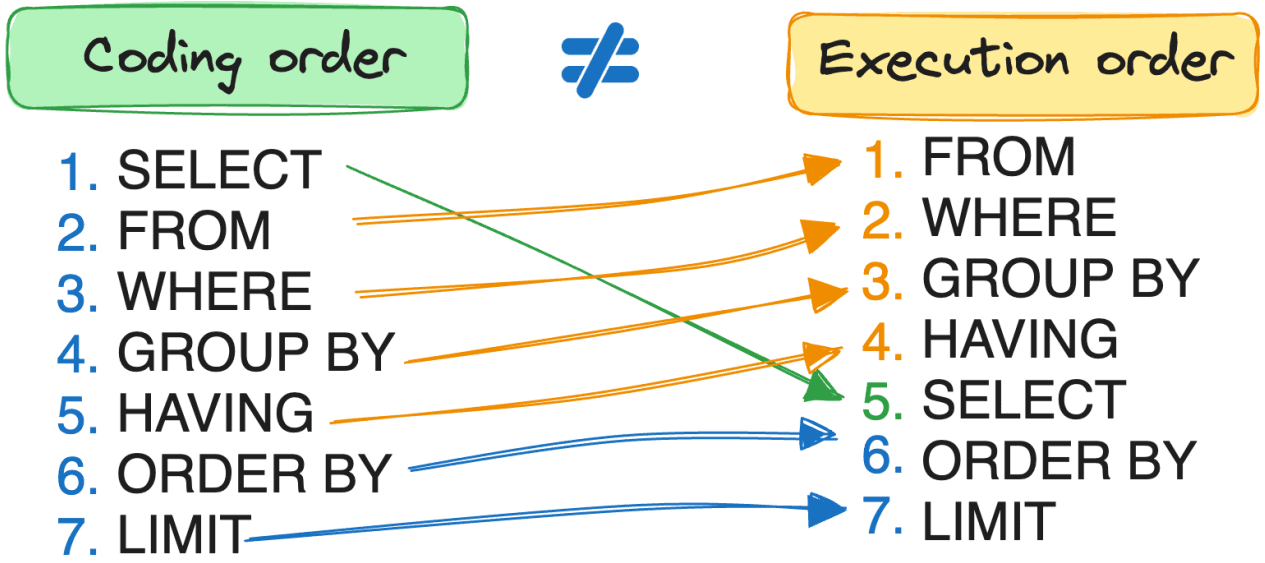
1. SQL Syntax and Order of Execution
2. Filtering Data
3. Sorting Results
4. Combining Tables

SQL Syntax Basics

- SQL is **case insensitive** for keywords (SELECT vs select)
- *Convention* is to write keywords in UPPERCASE
- Table and column names **are often case sensitive** depending on **database**
- Statements usually end with a semicolon ;
- Comments start with -- for single line
- Comments start with /* and end with */ for multiple lines

Order of Execution in SQL

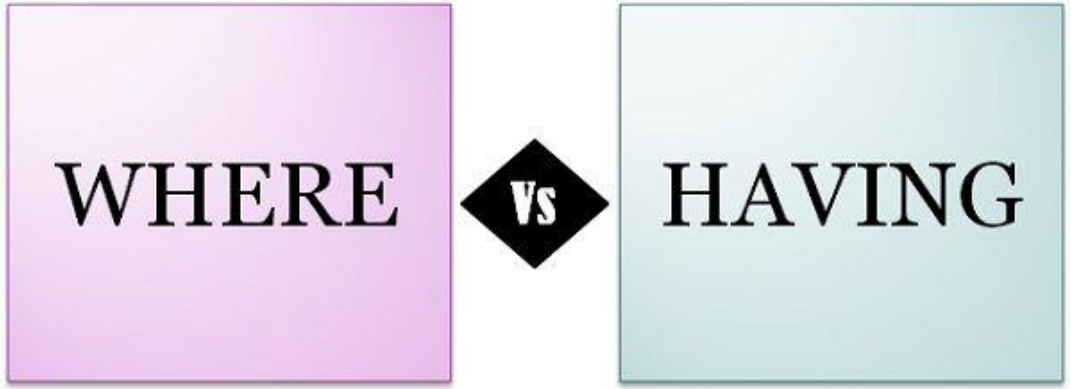
- Understanding order helps debug errors
  - FROM and JOINs
  - WHERE
  - GROUP BY
  - HAVING
  - SELECT
  - ORDER BY
  - LIMIT
- You cannot use column aliases in WHERE clause



https://www.kdnuggets.com/wp-content/uploads/ferrer_essential_guide_sql_execution_order_3.png

Why Execution Order Matters

- You cannot filter on an *aggregate* in **WHERE**
- You must use **HAVING** for *aggregate* filters
- You cannot use **SELECT** alias in **WHERE**
- Example Error: **SELECT SUM**(amount) AS total **WHERE** total > 100
- Correct: **SELECT SUM**(amount) AS total **HAVING** SUM(amount) > 100



https://techdifferences.com/wp-content/uploads/2017/12/intro-image.jpg

Summary of Key Commands

- **SELECT**: Choose columns
- **FROM**: Choose tables
- **WHERE**: Filter rows
- **GROUP BY**: Aggregate rows
- **HAVING**: Filter aggregates
- **JOIN**: Combine tables
- **ORDER BY:** Sort results

https://www.freepik.com/free-photos-vectors/important-png

Filtering Data (WHERE)

- **WHERE** clause filters rows before aggregation
- It is essential for isolating specific financial periods
- It isolates specific accounts or transaction types
- Common operators: =, <, >, <=, >=, <>
- Example: **WHERE** amount > 1000

Filtering Data (WHERE)

- Use single quotes for date strings: '2023-01-01'
- Operator **BETWEEN** is useful for ranges
- Example: **WHERE** transaction_date **BETWEEN** '2023-01-01' **AND** '2023-03-31'
- Be aware of time stamps including hours and minutes

Filtering Lists of Items (IN)

- Used when filtering for multiple specific values
- Common for account codes or currency types
- Example: **WHERE** currency **IN** ('USD', 'EUR', 'GBP')
- Cleaner than writing OR multiple times
- Improves readability of audit trails

Pattern Matching (LIKE)

- Used for text search in descriptions or memos
- Percent sign **%** represents any sequence of characters
- Underscore _ represents a single character
- Example: **WHERE** description **LIKE** '%Interest%'
- Useful for finding specific transaction types in ledgers

Handling Missing Data (NULL)

- **NULL** represents missing or unknown data
- It is not zero and it is not an empty string
- Cannot use '= **NULL'** to check for nulls
- Must use **IS NULL** or **IS NOT NULL**
- Example: **WHERE** closing_balance **IS NOT NULL**

Logical Operators in Filtering

- **AND** requires both conditions to be true
- **OR** requires at least one condition to be true
- **NOT** negates a condition
- Use parentheses to control logic order
- Example: **WHERE** (amount > 1000 **AND** currency = 'USD') **OR** flag = 'Review'

Sorting Data (ORDER BY)

- Orders the result set by one or more columns
- Default order is Ascending (**ASC**)
- Descending order uses **DESC**
- Critical for finding top expenses or largest transactions
- Example: **ORDER BY** amount **DESC**

Sorting by Multiple Columns

- You can sort by secondary columns if first matches
- Useful for grouping by date then by amount
- Example: **ORDER BY** transaction_date **ASC**, amount **DESC**
- Ensures consistent reporting order
- Helps in spotting anomalies chronologically

Limiting Results (LIMIT / TOP)

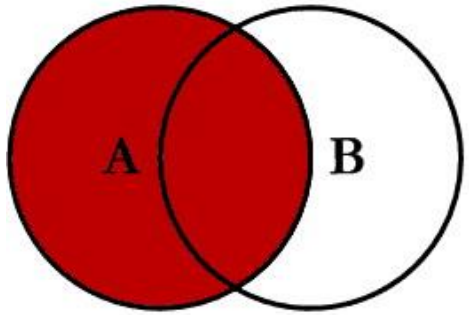
- Used to view only the first N rows
- Helps in previewing data without loading everything
- Syntax varies by dialect (LIMIT vs TOP)
- Example: **SELECT** * **FROM** transactions **ORDER BY** amount **DESC** **LIMIT** 10
- Finds the top 10 largest transactions quickly

Introduction to Joins

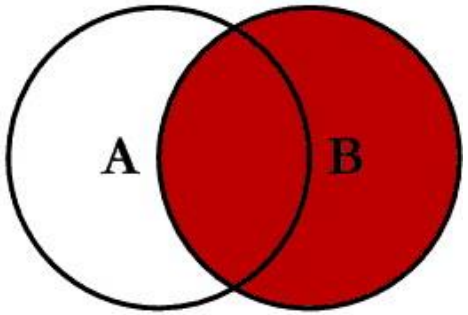
- Data is often split across multiple tables
  - Transactions in one table
  - Account Details in another
- Joins combine these tables based on a related column
- Critical for enriching transaction data with metadata
- Uses **Primary Keys** and **Foreign Keys**

https://pngtree.com/so/golden-key

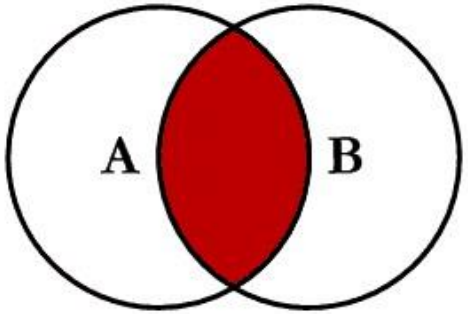
SQL JOINS



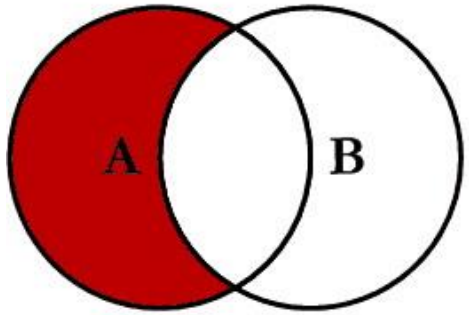

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



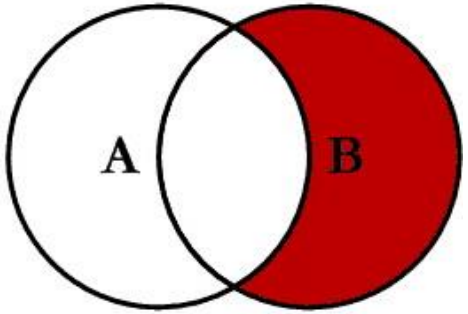

```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



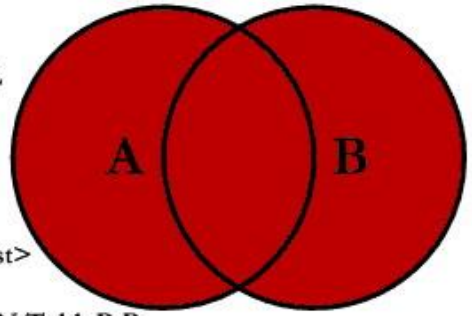

```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



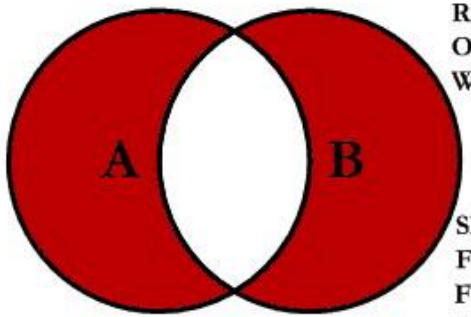

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```



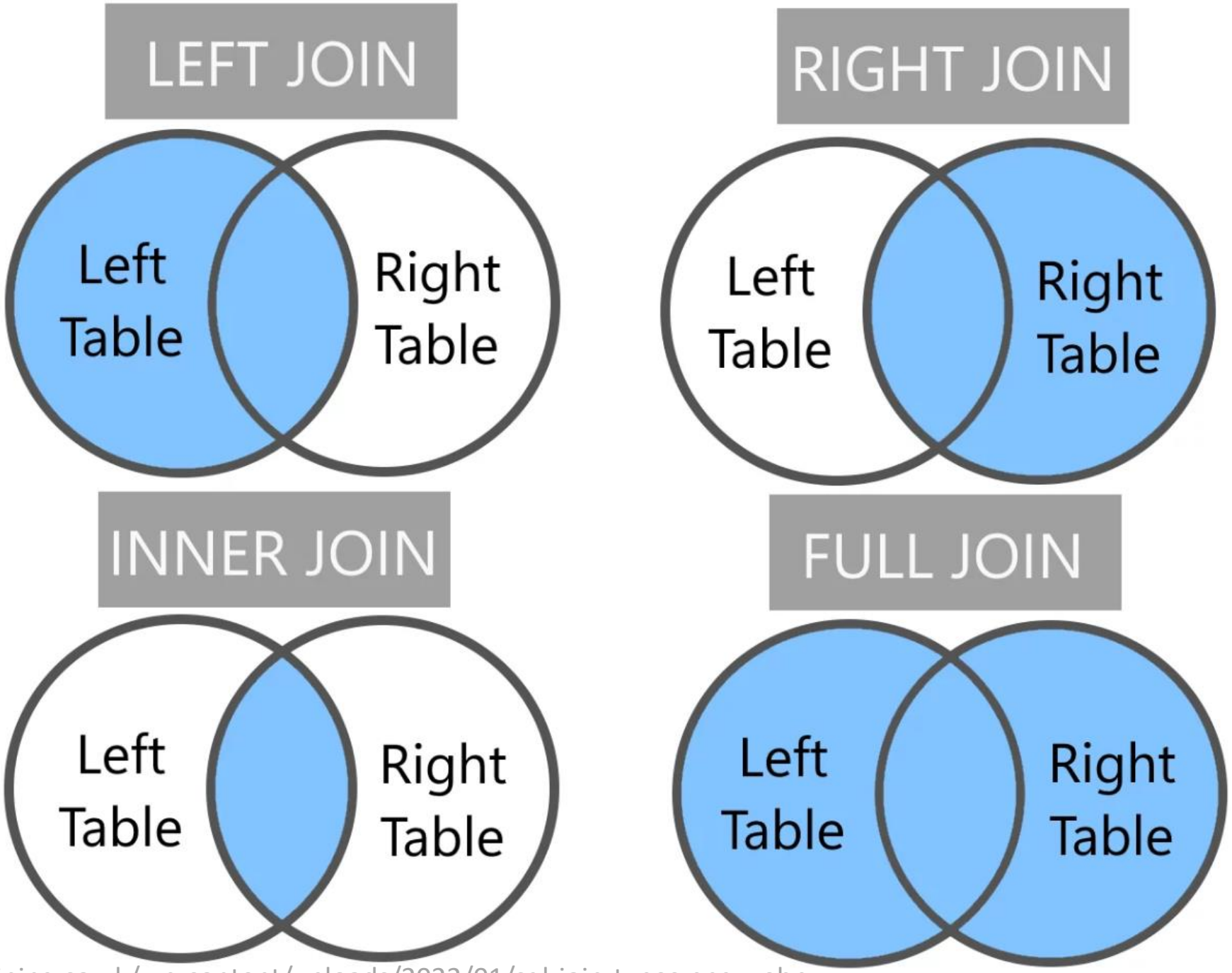

```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

© C.L. Moffatt, 2008

https://imgur.com/sql-joins-explained-1m55Wqo



https://www.acuitytraining.co.uk/wp-content/uploads/2022/01/sql-join-types.png.webp

Primary and Foreign Keys

- **Primary Key:** Unique identifier for a row in a table (e.g., Account_ID)
- **Foreign Key:** Column in another table referencing the Primary Key
- Join condition matches these keys
- Ensures data integrity across the database
- Example:

Transactions.Account_ID matches Accounts.Account_ID

INNER JOIN

- Returns **only rows** where there **is a match in both tables**
- If a transaction has no matching account, it **is excluded**
- *Most common* join for standard reporting
- Example: **SELECT** * **FROM** transactions **INNER JOIN** accounts **ON** transactions.id = accounts.id
- Ensures all data returned is valid in both systems

LEFT JOIN (or LEFT OUTER JOIN)

- Returns all rows from the left table
- Returns matching rows from the right table
- If no match, **NULL** values are returned for right table columns
- Useful for finding transactions without assigned accounts
- Example: **SELECT** * **FROM** transactions **LEFT JOIN** accounts **ON** transactions.id = accounts.id

RIGHT JOIN and FULL JOIN

- **RIGHT JOIN**: Opposite of **LEFT JOIN** (all from right table)
- **FULL JOIN**: All rows from both tables (matches and non-matches)
- Less common in daily financial analysis
- Useful for reconciliation tasks (finding mismatches)
- Most analysts rely on **INNER** and **LEFT** joins

Table Aliases

- Shortens table names for readability
- Essential when joining multiple tables
- Defined after table name using AS or just space
- Example: **FROM** *transactions t* **JOIN** *accounts a* **ON** t.id = a.id
- Columns referenced as **t.**amount or **a.**currency

Joining Multiple Tables

- You can join more than two tables in one query
- Chain **JOIN** statements sequentially
- Example: Transactions **JOIN** Accounts **JOIN** Branches **JOIN** Regions
- Be mindful of performance with many joins

Duplicate Rows in Joins

- Joins **can create duplicate** rows if relationships are not one-to-one
  - Example: One account might have multiple transactions
  - This is expected behavior for transactional data
- However, joining two tables with one-to-many on both sides creates explosion
- Always verify row counts after joining

Common Mistake: Forgetting Join Conditions

- Always specify the **ON** clause
- Missing **ON** creates a Cartesian Product:
  - Every row in table A matches every row in table B
- Result is massive duplicate data
- Crashes systems and gives wrong financial totals

Common Mistake: Data Types in Joins

- Ensure joining columns have same data type
- Joining Integer to Varchar may fail or be slow
- Check Account_ID is same type in both tables
- Implicit conversion can hide errors
- Verify schema before writing joins

Common Mistake: NULLs in Joins

- **NULL** does not equal **NULL** in SQL
- Rows with **NULL** keys will not match in **INNER JOIN**
- They may appear in **LEFT JOIN** as unmatched
- Be careful when joining on columns that allow **NULLs**
- Clean data beforehand if possible